CRM Design Patterns Implementation Report

Bridge Pattern

Case 1 Decoupling Notification from CommunicationChannel

Objective

Decouple abstraction (Notification) from implementation (CommunicationChannel)

Design Overview

- Abstraction: `Notification`
- Implementor: `CommunicationChannel`
- Concrete Implementors:
  - `ChatChannel`
  - `EmailChannel`
  - `SMSChannel`

Before vs After

Before Using Bridge (The Problem)
In a notification system, the application needs to support multiple notification types (like alerts, reminders) and multiple communication channels (like Email, SMS, Chat).

Without using the Bridge pattern, each notification type is tightly coupled with a specific channel.
The system contains classes like:
ChatNotification
EmailNotification
SmsNotification

If the system needs to support more combinations, it quickly results in a large number of classes.

Problems:

1. Class Explosion
2. Poor Scalability
3. Violation of Open-Closed Principle
4. Tight Coupling

After Using Bridge (The Solution)
By introducing the **Bridge pattern**, we separate:

Abstraction → Notification (WHAT to send)
Implementation → CommunicationChannel (HOW to send)

Instead of inheritance, we use composition so we can now mix any notification type with any channel at runtime.

How it works:

- Notification holds a channel
- Uses a general Notification class
- Channels implement `sendMessage()`
- Channel injected at runtime

Example:
`new ChannelNotification("Hello", new EmailChannel())`

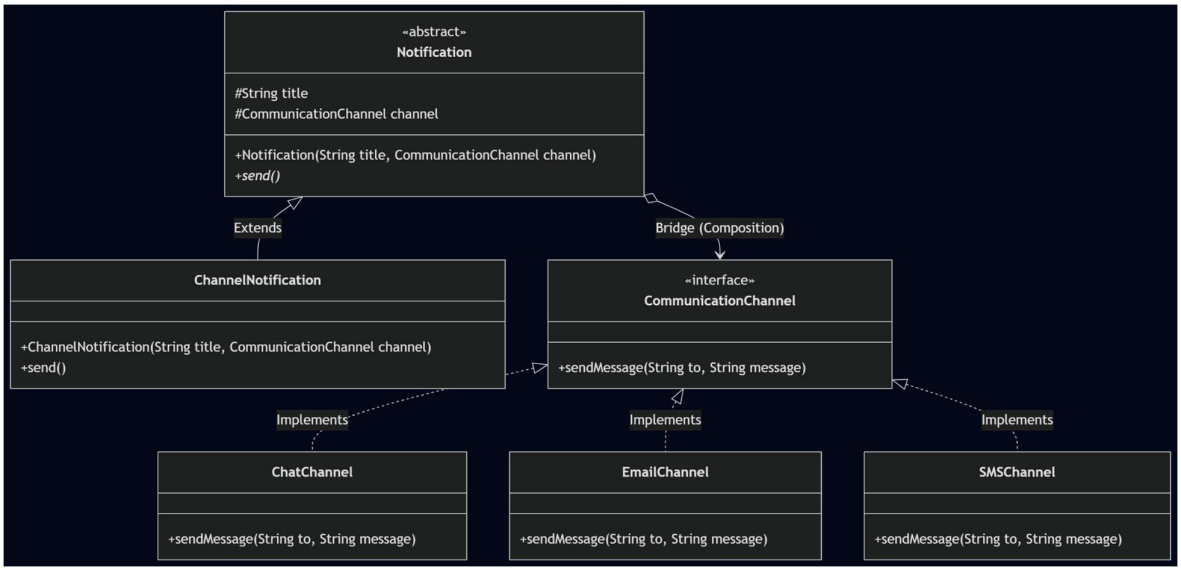
What Happens Now:

- No new classes for combinations
- Reuse same structure
- Runtime flexibility

Implementation Steps

1. Modify `Notification` class
2. Add `CommunicationChannel` field
3. Update constructor
4. Delegate sending logic
5. Update factories

UML Diagram



Benefits

Reduced class explosion
Easier extension
Better maintainability
Loose coupling
Cleaner design

Case 2 — Report Generation and Formatting

Objective

Separate report logic from rendering format

Design Overview

- Abstraction: `Report`
- Implementer: `ReportRenderer`
- Concrete Renderers:
  - Console
  - CSV
  - PDF

Before vs After

Before Using Bridge (The Problem: Class Explosion)
Each report type is tied to each format:
SalesPDFReport, SalesHTMLReport, PerformancePDFReport…

M × N class explosion

- To create an OrderReport in both Console and CSV formats, we would have needed separate subclasses: ConsoleOrderReport and CsvOrderReport.
- Adding a new report (e.g., InventoryReport) or a new format (e.g., PdfRenderer) would cause the classes to grow exponentially (M reports × N formats).

After Using Bridge (The Solution: Decoupling)
Report delegates rendering to a Renderer.

Now we can mix any report type with any render type:

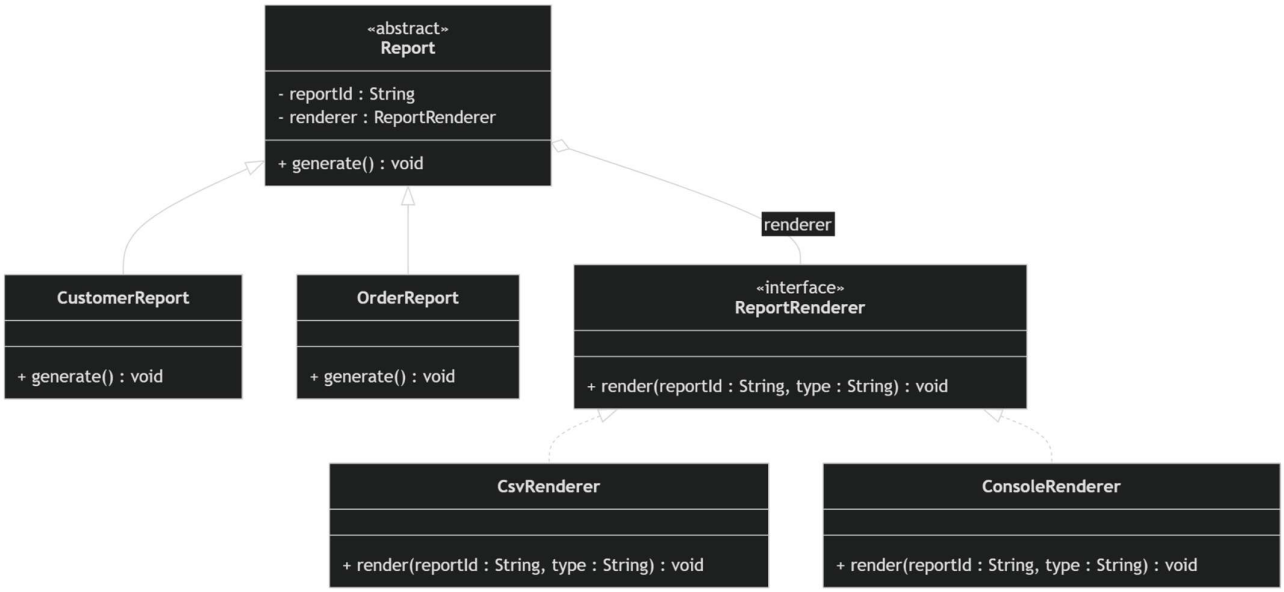
- SalesReport + PDFRenderer
- PerformanceReport + CSVRenderer

Only M + N classes

Implementation Steps

1. Create `ReportRenderer` interface
2. Implement concrete renderers
3. Modify `Report` class
4. Inject renderer via factory

UML Diagram



Benefits

Flexible output formats
Avoids duplication
Open for extension

Adapter Pattern

Case Study 1: External Payment Gateway (Stripe Adapter)

Objective

Adapt Stripe API to `PaymentProcessor`

Design Overview

- Target: `PaymentProcessor`
- Adapter: `StripePaymentAdapter`
- Adaptee: `StripeClient`

Before vs After

Before Using Adapter (The Problem)
CRM uses:

- Money object
- processPayment()

Stripe requires:

- Amount in cents
- Separate currency
- Method `charge()`

→ Incompatible APIs
→ Tight coupling risk

After Using Adapter (The Solution)
StripePaymentAdapter:

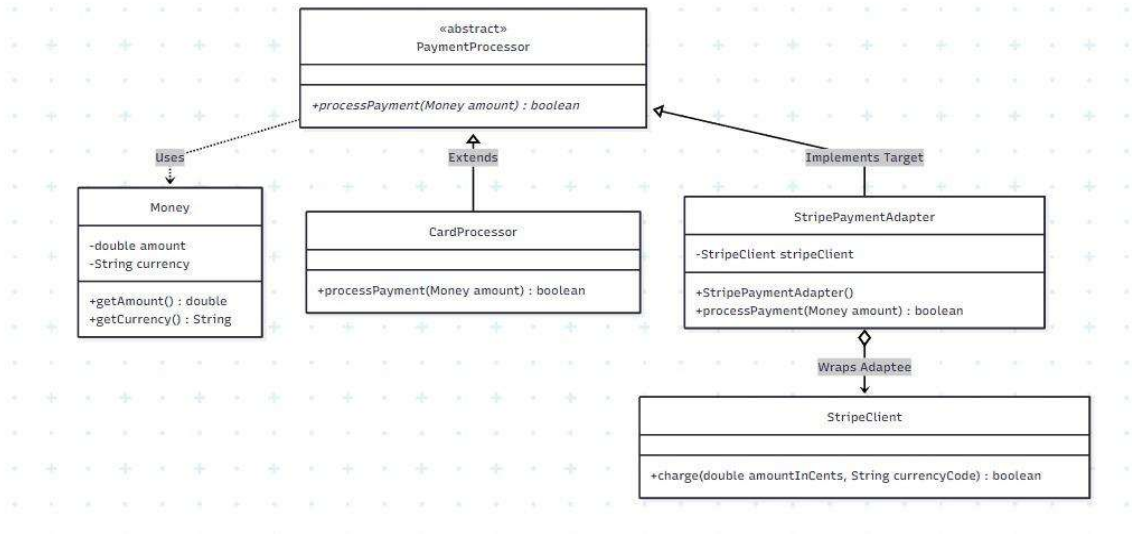
- Implements PaymentProcessor while wrapping StripeClient
- Converts data automatically

CRM calls:
`processPayment(money)` → Adapter translates to Stripe

Implementation Steps

1. Create adapter class
2. Wrap Stripe client
3. Translate method calls
4. Update factory

UML Diagram



Benefits

Integrates external APIs easily
Keeps system consistent
Decouples core logic

Case 2 — ERP System Integration (Supplier Adapter)

Objective

Integrate external ERP system

Design Overview

- Target: `SupplierService`
- Adapter: `ErpSupplierAdapter`

Before vs After

Before (The Problem):
Only local data

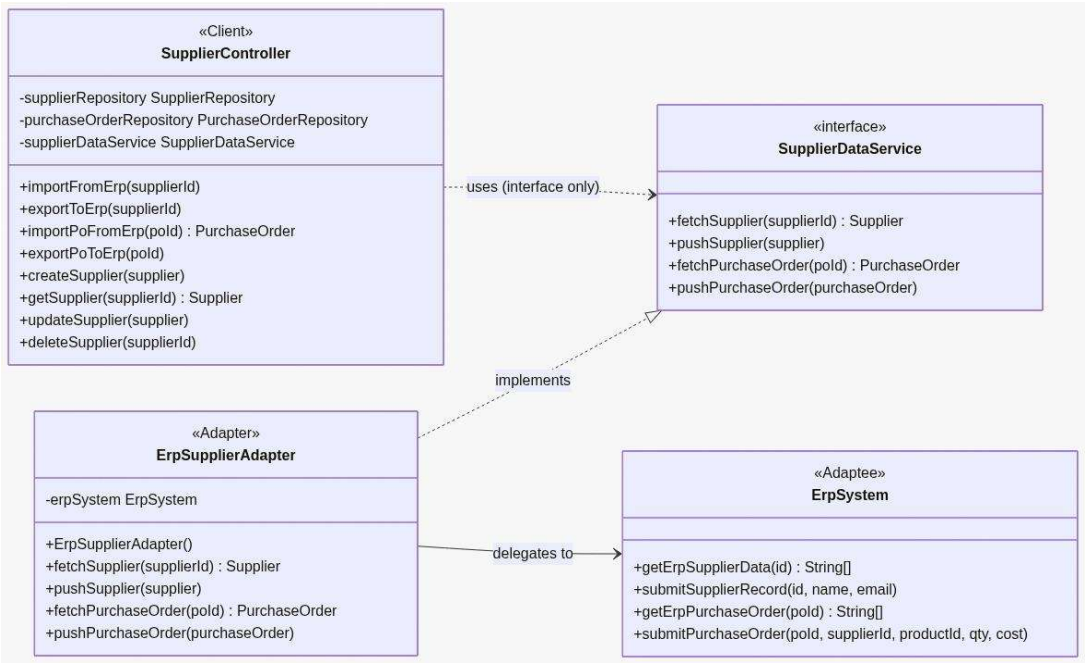
After (The solution):
ERP integrated via adapter

- The ErpSupplierAdapter maps raw ERP arrays into domain-driven Supplier and PurchaseOrder objects.
- The adapter maps ERP methods like getErpSupplierData() to the CRM's fetchSupplier() interface.

Implementation Steps

1. Define service interface
2. Implement adapter
3. Update controller usage

UML Diagram



Benefits

External system integration
Decouples business logic

Flyweight Pattern

Case— Product Cache

Objective

Share Product instances

Design Overview

- Flyweight: `Product`
- Factory: `ProductCache`

Before vs After

Before Using Flyweight (The Problem)
10,000 orders → 10,000 identical Product objects

Problems:

- High memory usage
- Performance overhead
- Inconsistent updates

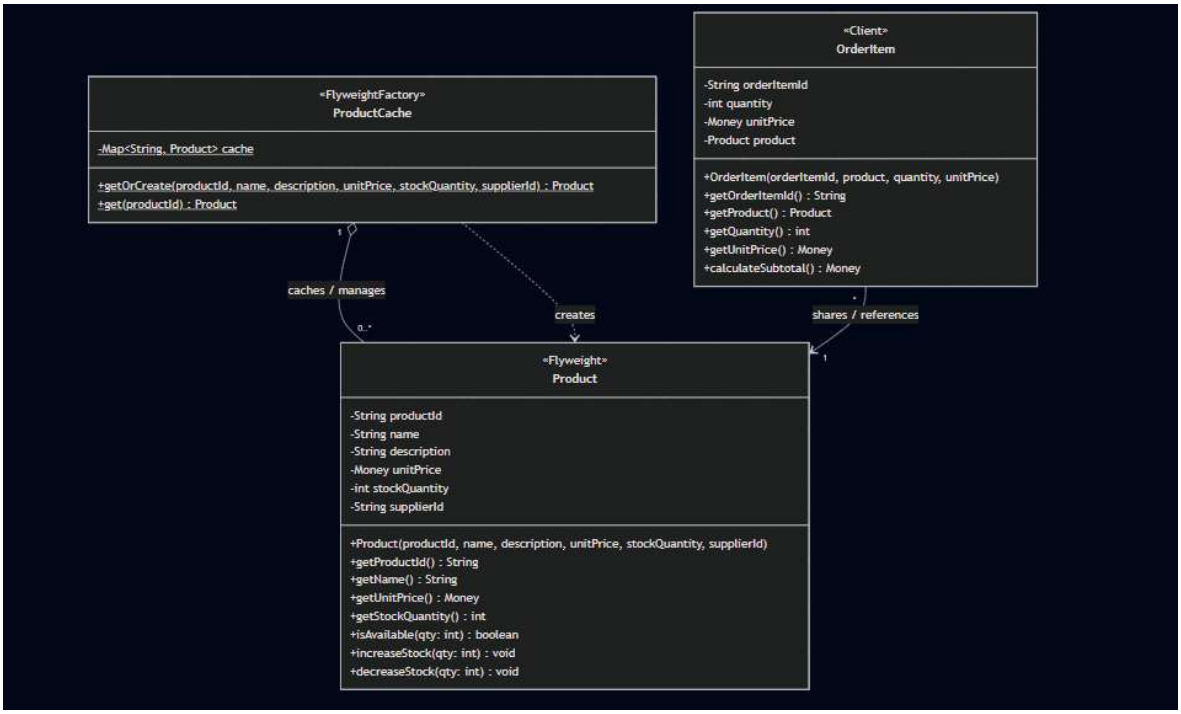
After Using Flyweight (The Solution)
Using ProductCache:

- First request → create object
- Next requests → reuse same object

Implementation Steps

1. Make Product immutable
2. Create cache
3. Use `getOrCreate()`

UML Diagram



Benefits

Reduced memory usage
Improved performance
Single source of truth

Decorator Pattern

Case 1 — SystemAlert Enhancements

Objective

Add behaviors dynamically

Design Overview

- Component: `SystemAlert`
- Decorator: `AlertDecorator`
- Concrete Decorators:
  - Logging
  - Retry
  - Escalation

Before vs After

Before Using Decorator (The Problem)
Inheritance leads to:

- Class explosion
- No runtime flexibility

After Using Decorator (The Solution)

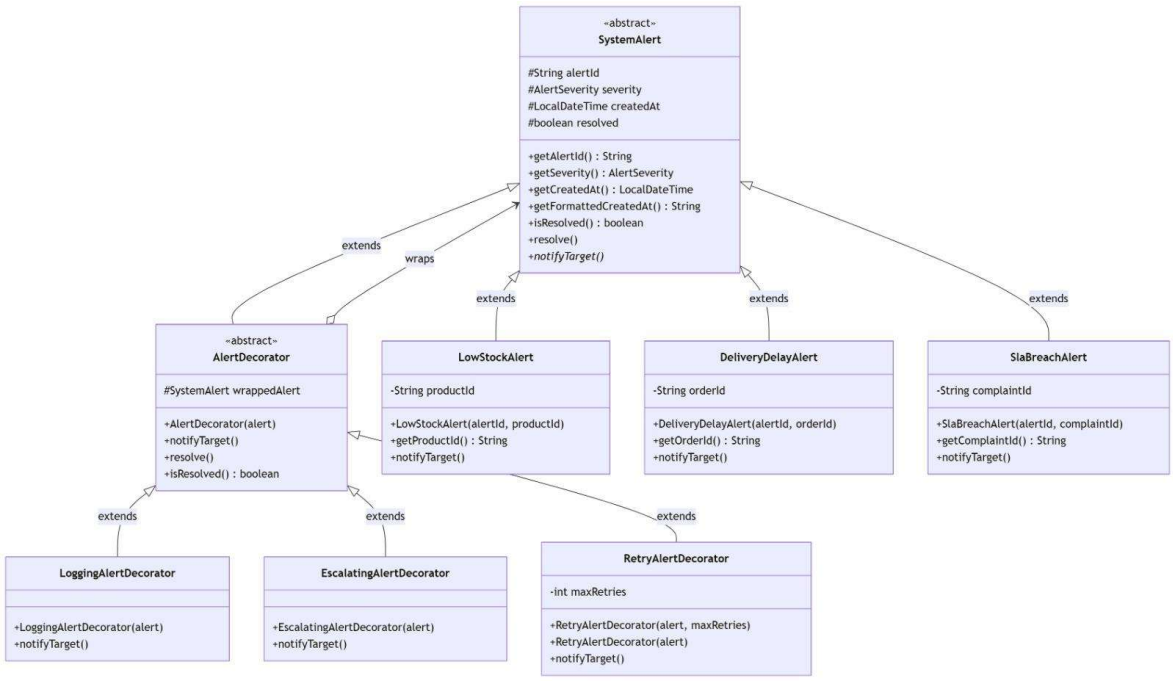
- Each decorator handles its specific logic then delegates the notification to the wrapped alert.
- Dynamic wrapping at runtime:

```
new LoggingAlertDecorator(new RetryAlertDecorator(alert))
```

Implementation Steps

1. Create base decorator
2. Implement concrete decorators
3. Stack decorators

UML Diagram



Benefits

Flexible behavior extension
No subclass explosion

Case 2 — Delivery Status Tracking

Objective

Add logging, notifications, alerts

Design Overview

- Component: `Delivery`
- Decorator: `DeliveryDecorator`
- Concrete Decorators:
  - Logging
  - Notification
  - Alerting

Before vs After

Before Using Decorator (The Problem)

If we wanted to automatically trigger a `DeliveryDelayAlert` when a shipment is delayed, we would have been forced to modify the core `Delivery` class

- Polluted Delivery class
- SRP & OCP violation
- Class explosion with inheritance

After Using Decorator (The Solution)

- **The `AlertingDeliveryDecorator` adds alerting logic while the original `Delivery` class remains responsible only for state.**
- Dynamic wrapping choose to wrap only high-priority deliveries with alerting features at runtime:

```
new AlertingDeliveryDecorator(newLoggingDeliveryDecorator(delivery))
```

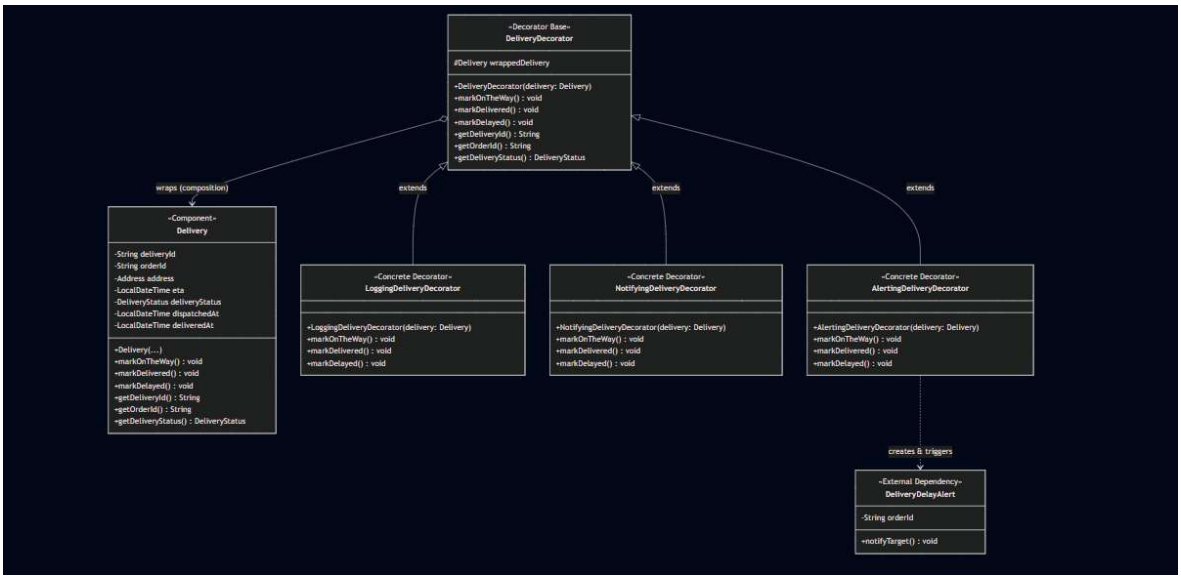
Flow:

- Alert → Logging → Delivery

Implementation Steps

1. Create decorator base
2. Implement behaviors
3. Wrap delivery object

UML Diagram



Benefits

Clean separation of concerns
Extensible behavior
Flexible combinations